

Measurement of Similarity between Use Case Description and Sequence Diagram in Software Requirement Specification using Text Analysis for Dtrain Application

1st Rifqi Zaidan Irsyad Yanis

Department of Informatics

Telkom University

Bandung, Indonesia

rifqiyanis(at)student.telkomuniversity.a
c.id

2nd Yudi Priyadi

Department of Software Engineering

Telkom University

Bandung, Indonesia

whyphi(at)telkomuniversity.ac.id

3rd Shinta Yulia Puspitasari

Department of Software Engineering

Telkom University

Bandung, Indonesia

shintayulia(at)telkomuniversity.ac.id

Abstract—Dtrain application is built by implementing software documents, one of which is the Software Requirement Specification (SRS). In compiling software documents, there is a mismatch between one document element with other elements. Therefore, this study will measure the similarity between the Use Case Description (UCD) on the SRS case study Dtrain with Sequence Diagrams (SD) to determine how similar or compatible the two SRS elements are.

This study aims to measure the similarity of UML artifacts between UCD and SD contained in a software development document. The suitability of the two artifacts can be done by analyzing the text in the description (UCD) and object (SD) sections.

Based on the results and discussion, this study resulted in the following. First, a study that finds level of similarity between UCD and the SD by comparing the step results performed an extraction on the UCD and the extraction of messages and actors on the SD. Second, there are five use cases in the Dtrain application, so the UCD and SD are each 5. Third, the highest similarity value results are found in SD 1 and SD 5, with a similarity value of 0.712. Fourth, the results of the validation value using algorithm code to get a kappa score of 0.331. Fifth, the results of the validation obtained from results of the expert questionnaire got a value of 0.8905.

Keywords—Similarity, Software Requirement Specification, Validity, Text Analysis, Use Case.

I. INTRODUCTION

A software Requirements Specification (SRS) document is defined as a file that contains software requirements [1]. SRS has many objects, descriptions, or diagrams that represent the requirements of the software. To help work on the stages of the software development cycle, documents are made so that each stage and work can be documented, more focused, and a source of information for application developers or people who will use the application (user). Examples of these documents are Software Testing Documentation (STD) as a document containing documentation related to software testing, Software Design Description (SDD) as a document containing an explanation of software design from the user interface to algorithms, and other documents. According to application development needs [1, 2, 3].

Every application development requires an SRS document, also for Dtrain application. Dtrain is a web-based application with the primary function of booking train tickets. This application is built by implementing software documents, one of which is SRS. The documents and source

code of the Dtrain application are documents and source code placed in a public code repository so that the documents and source code are open sources.

In compiling software documents, there is a mismatch between one document element with other elements, Likewise with the preparation of SRS documents on the Dtrain application. Therefore, this study will measure the similarity between the Use Case Description on the SRS case study Dtrain with Sequence Diagrams to determine how similar or compatible the two SRS elements are.

This study aims to measure the similarity of UML artifacts between Use Case Description (UCD) and Sequence Diagrams (SD) contained in a software development document. The suitability of the two artifacts can be done by analyzing the text in the description (UCD) and object (SD) sections.

In this study, there are novelty and contributions as follows:

1. Analyzing the text contained in the UCD description through text preprocessing.
2. Calculate the similarity of the text description analysis results with the object in the Sequence Diagram.
3. Gwet's AC1 formulation is used to test the validity and reability of the similarity results.

II. RELATED WORKS

A. Software Requirement Specification (SRS)

Software Requirement Specification or commonly abbreviated as SRS, is a document that contains requirements or descriptions of system requirements of a software [1]. The SRS document is essential for working on a software development project. Through SRS, software development can be more focused because SRS is also a road map that provides direction to all parties involved in the project so that the results meet the requirements of users' needs.

In making software, the SRS document must make it easier for software developers or anyone reading to understand the Requirement Specification [1, 4]. Therefore, there are many templates or SRS that can be used in creating SRS documents.

SRS has functional requirements, non-functional requirements, use cases, diagram of use cases, description of use cases, sequence diagrams, and other elements.

B. UC Diagram and UC Description

Use case diagrams are several Unified Modeling Language (UML) diagrams that describe the interaction relationships between systems and actors. Use cases can describe interactions between system users and the system [5]. The components of a use case diagram are the system as the boundary of the interaction of the use case, the actors representing people, and the use case, the interaction between the system and the actors [6].

Use Case Description is a description that describes the processing details of the use case [7]. This document is part of an SRS element that provides an overview of business process functionality, including system creation. Some components of a use case description are use case names, scenarios, actors, and activity streams [8, 9].

C. Sequence Diagram

A *sequence diagram* is a diagram that is used to explain and display the interactions between objects in a system in detail [10]. In addition, the sequence diagram will also display the message or command that was sent, along with the execution time. Objects related to the operation process's running are usually sorted from left to right [10, 11]. Some components in the sequence diagram are actors, lifelines, activation boxes, objects, and messages [5].

D. Text Analysis dan Text Pre-Processing

Text analysis, also known as text mining, refers to data mining in textual form, where the data source is usually obtained from documents. The aim is to find words describing the documents' content to perform a connectivity analysis [12, 13, 14, 15].

Text preprocessing is one of the steps of the text mining method. Text preprocessing consists of several stages: case folding, tokenizing, filtering, and stemming [16, 17, 18, 19].

E. Cosine Similarity

Cosine similarity is a measure of similarity to two (2) vectors [9, 26]. Cosine similarity calculates the similarity of the cosine value between the angles of the two vectors [20]. *Cosine similarity* is a formula commonly used to calculate the similarity between two data, in this case, the data from text analysis results. The formula for cosine similarity is as follows:

$$\text{similarity}(A,B) = \frac{A \cdot B}{\|A\| \times \|B\|} = \frac{\sum_{i=1}^n A_i \times B_i}{\sqrt{\sum_{i=1}^n A_i^2} \times \sqrt{\sum_{i=1}^n B_i^2}} \quad (1)$$

F. Gwet's AC1

Gwet's AC1 is a statistical, computational procedure that assesses the reliability and effectiveness of a recommendation model by calculating the index of agreement between two experts or between experts. [21, 22, 23, 24]. Table I shows the validation results obtained from the κ -scores for grouping purposes.

TABLE I. GWET'S AC1 VALUE INDEX	
Kappa Statistic	Strenght of Agreement
0.81 – 1.0	Very good
0.61 – 0.8	Good
0.41 – 0.6	Moderate
0.21 – 0.4	Fair
<0.2	Poor

III. METHODOLOGY

As a reference for the stages of activity in this research, Figure 1 shows stages of research flow, which are presented in the form of a structured chart.

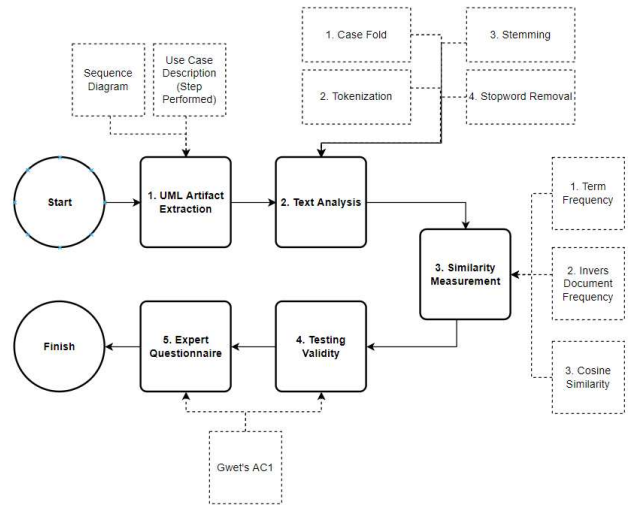


Fig. 1. Research Flow

The following are the steps taken in completing the research, namely:

1. UML Artifact Extraction.

The first modeling process is to check and complete the parts of the SRS case study application (Dtrain) so that the next stage can be appropriately implemented. After checking the parts, extraction is carried out on the step performed from UC Description and message and actor form SD.

2. Text Analysis.

Text Analysis is conducted from the artifact modeling results so that the results can be used to look for similarities. There are four main stages carried out in carrying out this process: do case folding, tokenization, stemming, and stopwords.

3. Similarity Measurement

This process uses the cosine similarity formula to obtain similarity results from the data set processed by the text preprocessor. The cosine similarity calculation is performed using the Python algorithm on the text analysis results. The main phases of similarity measurement are term frequency shaping and cosine similarity measurement.

4. Testing Validity.

Tests are carried out to determine whether the results obtained can be trusted. Validity is calculated using Gwet's AC1 formula and produces Kappa index.

5. Expert Questionnaire.

Comparing the value of the validity of the python calculation with the results of the equation questionnaire conducted by two experts. divided into three stages, namely calculating the observed agreement, calculating the chance-agreement probability, and AC1 statistics.

IV. RESULT AND DISCUSSION

A. UML Artifact Extraction

In this study, as the first step of doing the analysis, the extraction of the two UML artifacts was carried out. The reference data set of similar processes from UCD and SD is contained in the "step performed" section of UCD. In addition, for SD, use the "message" and object/actor sections of SD.

Five use case extractions of Dtrain application SRS are as follows:

1. User inputs username and password into the system, the system checks the username and password from the database system, the system displays the dashboard page to the user if the username and password match.
2. The user inputs personal data on the train ticket booking system, the user presses the register button on the train ticket booking system, the system saves data to the database, the system displays the login form to the user.
3. The user opens the ordering menu on the system, the system displays the order input form display on the application, the user enters the order data in the system, the system displays the train form to the user, the user enters the train data in the system, the system displays the booking form to the user.
4. The user opens the pay menu in the application, The system displays the payment method page to the user, The user selects the payment method in the application, The system displays the booking code to the user, Users make payments on the application, Admin provides payment from the system to the user, The system verifies and stores user payment data in the application, The system displays the verified payment results to the user User receives valid proof of payment via email from the system.
5. The user logs in via Google on the application, The system displays the google account display to the user, The user logs in to a google account on the system, The google system redirects the user to the application dashboard, The user enters the application dashboard.

In the next stage, extraction for the Sequence Diagram is also carried out with the following results:

1. Buyer opens login window, inputs login data, clicks login button, system checks login information.
2. Buyer opens registration window, inputs registration data, clicks register button, system checks registration data.
3. Buyer opens booking menu, clicks booking menu, inputs booking form, booking form submitted to database.
4. Buyer opens payment menu, chooses payment option, pays the invoice, information goes to database.

5. Buyer opens login tab, clicks google login button, redirects to google login menu, google checks the login information.

B. Text Analysis

After the UML extraction results are obtained, the text analysis process is carried out in the following stages:

1. Case Folding

This stage is carried out as the first stage in text analysis which aims to convert each word in the inputted document into a document that is all lowercase or capitalized into a standard form. The algorithm can be seen in figure 2.

```
[4] #case fold
    sentences = [line.lower() for line in elicitation]
    with open('uml_extract_casefolded.txt', 'w') as out:
        out.writelines(sorted(sentences))
    print(sentences)
```

Fig. 2. Case Folding

2. Tokenization

After the text document goes through the case fold stage, the tokenization stage is carried out, which groups the text into specific parts. Can be seen in Figure 3.

```
[6] #Tokenization
    LemDocuments = CountVectorizer(analyzer='word', stop_words='english')
    LemDocuments.fit_transform(dataset)
    print(LemDocuments.vocabulary_)
```

Fig. 3. Tokenization

3. Stemming

The text document is then converted into the basic word form in the stemming process. The process is carried out so that the affixes of each word in the text are lost so that each word turns into a basic word form. See Figure 4.

```
[7] #Stemming
    porter=PorterStemmer()

    def stemSentence(sentence):
        token_words=word_tokenize(sentence)
        token_words
        stem_sentence=[]
        for word in token_words:
            stem_sentence.append(porter.stem(word))
            stem_sentence.append(" ")
        return "".join(stem_sentence)

    for i in range(len(dataset)):
        dataset[i]=stemSentence(dataset[i])
    print(dataset[i])
```

Fig. 4. Stemming

4. Stopword Removal

In order to make the text that is searched for similarity more accurate, it is necessary to delete words that are common words that do not provide essential information in the text. The deleted stopwords are taken from the stopwords list contained in the "words" library. See Figure 5.

```
[12] #stopword removal
stop_words = set(stopwords.words('english'))
file1 = open("uml_extract_casefolded.txt")
line = file1.read()
words = line.split()

for r in words:
    if not r in stop_words:
        appendFile = open('stopwords_removed.txt', 'a')
        appendFile.write(" "+r)
        appendFile.close()

fo = open("stopwords_removed.txt", "r+")
string = fo.read()
print("STOP WORD REMOVAL :", string)
fo.close()
```

Fig. 5. Stopword Removal

C. Similarity Measurement

The similarity measurement process after text preprocessing is carried out is as follows:

1. Term Frequency (TF)

Before calculating the similarity, the term frequency matrix is formed, which is a matrix of the number of occurrences of words in the text. See Figure 6.

```
[8] #Term frequency (TF) Matrix Documents
tf_matrix = lenDocuments.fit_transform(dataset).toarray()
print(tf_matrix)

[[0 0 0 0 0 1 0 0 0 1 0 1 1 0 0 0 0 0 0 1 0 0 0 0 1 0 0 0 0 0 1 3 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 2 3 0 0 0 0 0]
 [0 0 0 2 1 0 0 0 0 0 0 2 1 1 0 0 1 0 0 0 1 0 0 1 0 0 0 0 0 0 0 0 0 0 0 1
 1 0 0 0 0 1 0 0 1 0 0 0 0 0 2 2 3 0 0 0 0 0]
 [0 0 1 1 0 0 0 0 0 0 0 2 0 4 0 2 3 0 0 0 1 0 0 0 0 0 1 0 1 0 3 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 2 5 0 0 0 0 0]
 [0 1 4 1 0 0 0 0 0 1 0 1 0 3 1 0 0 0 0 0 0 0 0 1 0 1 2 1 0 0 1 0 1 7 0
 0 1 1 1 0 0 0 1 0 1 1 0 0 0 0 7 0 1 1 2 0]
 [2 0 3 0 0 0 0 0 0 2 0 0 2 0 1 0 0 4 0 0 0 2 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 5 0 0 0 0 0]
 [0 0 0 0 1 1 1 0 1 0 0 1 0 0 0 0 0 0 0 1 1 0 0 4 0 0 0 0 1 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1]
 [0 0 0 0 1 1 1 0 1 0 0 2 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0
 0 0 0 0 1 3 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1]
 [0 0 0 4 0 1 0 0 1 0 0 0 1 0 0 0 2 0 0 0 1 0 0 0 0 0 0 2 0 1 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0]
 [0 0 0 0 0 1 0 1 0 0 0 0 0 1 0 0 0 0 1 0 1 0 1 0 0 0 0 1 0 1 1 0 0 0 1 2 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
 [0 0 0 0 1 1 1 0 1 0 0 0 0 0 0 0 0 0 3 1 0 0 0 4 0 0 1 0 1 0 0 0 0 0 0 0
 0 0 0 1 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0]]
```

Fig. 6. Term Frequency

2. Invers Document Frequency (IDF)

The stage after forming term frequency is to map invers document frequency value to term frequency matrix. See Figure 7.

```
[24] #TF-IDF matrix result
vect = TfidfVectorizer(min_df=1, stop_words="english")
tfidf = vect.fit_transform(dataset)

#calculate cosine similarity
cosine_similarity = (tfidf * tfidf.T).toarray()
print(cosine_similarity)

[[1. 0.14934599 0.17650034 0.1485096 0.18778471 0.04215782
 0.04003764 0.04103492 0.02927132 0.02028219 ]
 [0.14934599 1. 0.51031336 0.24264962 0.21671329 0.23139058
 0.17242532 0.32947533 0.03198973 0.13433199]
 [0.17650034 0.51031336 1. 0.35814138 0.38497962 0.05648272
 0.08437279 0.28130029 0.03119695 0.02161615]
 [0.1485096 0.24264962 0.35814138 1. 0.36056335 0.01772415
 0.02688908 0.07867711 0.39539308 0.01414734]
 [0.18778471 0.21671329 0.38497962 0.36056335 1. 0.
 0. 0. 0. 0.33691121]
 [0.04215782 0.23139058 0.05648272 0.01772415 0. 1.
 0.27122412 0.10108456 0.10396828 0.7122897 ]
 [0.04003764 0.17242532 0.08437279 0.02688908 0. 0.27122412
 1. 0.09600087 0.05097198 0.1138045 ]
 [0.04103492 0.32947533 0.28130029 0.07867711 0. 0.10108456
 0.09600087 1. 0.1533607 0.10626257]
 [0.02927132 0.03198973 0.03119695 0.39539308 0. 0.10396828
 0.05097198 0.1533607 1. 0.10858072]
 [0.02028219 0.13433199 0.02161615 0.01414734 0.33691121 0.7122897
 0.1138045 0.10626257 0.10858072 1. ]]]
```

Fig. 7. Invers Document Frequency

3. Cosine Similarity

The similarity measurement was performed by comparing one sentence with every other sentence.

The highest similarity value was found between sentences 5 and 9 or the extraction of Sequence Diagram 1 with Sequence Diagram 5, which was worth 0.712. See Figure 8.

```
[13] print(pd.DataFrame(cosine_similarity))

      0      1      2      3      4      5      6  \
0  1.000000  0.149346  0.176500  0.148510  0.187785  0.042158  0.040038
1  0.149346  1.000000  0.510313  0.242650  0.216713  0.231391  0.172425
2  0.176500  0.510313  1.000000  0.358141  0.384980  0.056483  0.084373
3  0.148510  0.242650  0.358141  1.000000  0.360563  0.017724  0.026889
4  0.187785  0.216713  0.384980  0.360563  1.000000  0.000000  0.000000
5  0.042158  0.231391  0.056483  0.017724  0.000000  1.000000  0.271224
6  0.040038  0.172425  0.084373  0.026889  0.000000  0.271224  1.000000
7  0.041035  0.329475  0.281300  0.078677  0.000000  0.101085  0.096001
8  0.029271  0.031990  0.031197  0.395393  0.000000  0.103968  0.050972
9  0.020282  0.134332  0.021616  0.014147  0.336911  0.712290  0.113805

      7      8      9
0  0.041035  0.029271  0.020282
1  0.329475  0.031990  0.134332
2  0.281300  0.031197  0.021616
3  0.078677  0.395393  0.014147
4  0.000000  0.000000  0.336911
5  0.101085  0.103968  0.712290
6  0.096001  0.050972  0.113805
7  1.000000  0.153361  0.106263
8  0.153361  1.000000  0.108581
9  0.106263  0.108581  1.000000
```

Fig. 8. Cosine Similarity

D. Validity Test

Testing how valid the results are is done using Gwet's AC1 calculations. The validity process produces a kappa index with exact values : See Figure 9. It was found that the kappa score was 0.3315, which was categorized into the "Fair" value category.

```
[14] #Validation
from sklearn.metrics import cohen_kappa_score
cohen_kappa_score(tf_matrix[5], tf_matrix[7])

0.3315565031982942
```

Fig. 9. Validity Test

E. Expert Questionnaire Results

After calculating the similarity, which was validated with the python program, the similarity calculation was carried out through a questionnaire by two experts.

Table II shows the results of a survey completed by an expert who already has his UML experience and understanding of SRS documents.

TABLE II EXPERT QUESTIONNAIRE

Subject 1	Subject 2		
	Agree	Disagree	Total
Agree	40	0	40
Disagree	4	6	10
Total	44	6	50

Steps conducted in this process are calculating P-value, chance-agreement, then Aggregation Coefficient. The processes are conducted sequentially.

The first step is calculating the questionnaire results and the observed agreement value between the two experts based on the P-value in Table II.

$$P = \frac{40+6}{50} = 0.92 \quad (2)$$

After that is to calculate the value of e(Y) or the value of Chance-Agreement:

$$P_1 = \frac{(44+40)/2}{50} = 0.84 \quad (3)$$

$$e(V) = 2(0.84)(1 - 0.84) = 0.2688 \quad (4)$$

Next step is to compute the AC1 statistic using the Aggregation Coefficients.

$$AC1 = \frac{0.92 - 0.2688}{1 - 0.2688} = 0.8905 \quad (5)$$

The results of above is 0.8905, which was included in the "Very Good" category. If the two similarity calculations are compared, there is a difference; the python program code has a value of 0.331, and according to the questionnaire filled out by expert, 0.8905.

V. CONCLUSION AND FUTURE WORK

Based on the discussion of the research results, some points can be highlighted from the overall research to be used as conclusions, namely:

1. The degree of similarity between the use case description and the sequence diagram was calculated by comparing the steps in the use case diagram and the extraction results of the messages and actors in the sequence diagram.
2. There are five use cases in the Dtrain application, so the Use Case Description and Sequence Diagrams are 5.
3. The results of the highest similarity value are found in Sequence Diagram 1 and Sequence Diagram 5 equals 0.712.
4. The results of the validation value using algorithm code get a kappa score of 0.331.
5. The results of the validation value using Gwet's AC1 formula obtained from the results of the expert questionnaire got a value of 0.8905.

For Future work, it can be implemented by calculating similarity from each use case or replacing UML artifacts to measure similarity with two other different types of UML artifacts.

ACKNOWLEDGMENT

This research was conducted with Mr. Yudi and Mrs. Shinta as supervisors of the RPL Study Program at Univ. Telkom. May the kindness of the guides get a reward of happiness from Allah SWT. Amen.

REFERENCES

- [1] IEEE Computer Society. Software Engineering Standards Committee., & IEEE-SA Standards Board. (1998). IEEE recommended practice for software requirements specifications. Institute of Electrical and Electronics Engineers.
- [2] Computer Society, I., & Engineering Standards Committee, S. C. (2013). Software and systems engineering-Software testing-Part 3: Test documentation Ingénierie du logiciel et des systèmes-Essais du logiciel-Partie 3: Documentation des essais. www.iso.org
- [3] Engineering Standards Committee of the IEEE Computer Society, S. (2009). IEEE Std 1016-2009 (Revision of IEEE Std 1016-1998), IEEE Standard for Information Technology—Systems Design—Software Design Descriptions.
- [4] A. O. J. Sabriye and W. M. N. W. Zainon, "A framework for detecting ambiguity in software requirement specification," 2017 8th International Conference on Information Technology (ICIT), 2017, pp. 209-213, doi: 10.1109/ICITECH.2017.8080002.
- [5] J. M. Almendros, and L. Iribarne, "Describing use-case relationships with sequence diagrams," Computer Journal, 2007, 50(1), 116–128, https://doi.org/10.1093/comjnl/bxl053.
- [6] P. Samuel, R. Mall and S. Sahoo, "UML Sequence Diagram Based Testing Using Slicing," 2005 Annual IEEE India Conference - Indicon, 2005, pp. 176-178, doi: 10.1109/INDCON.2005.1590149.
- [7] Y. Seki, S. Hayashi, and M. Saeki, "Detecting bad smells in use case descriptions," 2019 Proceedings of the IEEE International Conference on Requirements Engineering, 2019-September, 98–108, https://doi.org/10.1109/RE.2019.00021.
- [8] R. T. Anchiêta, R. F. de Sousa and R. S. Moura, "Identifying of user interface elements from use case descriptions," 2012 XXXVIII Conferencia Latinoamericana En Informatica (CLEI), 2012, pp. 1-6, doi: 10.1109/CLEI.2012.6427184.
- [9] L. Erazo, E. Martins and J. G. Greggi, "MARITACA: From Textual Use Case Descriptions to Behavior Models," 2017 47th Annual IEEE/IFIP International Conference on Dependable Systems and Networks Workshops (DSN-W), 2017, pp. 83-90, doi: 10.1109/DSN-W.2017.33.
- [10] N. Apriyanto, Y. Priyadi and D. S. Kusumo, "Extraction of Step Performed in Use Case Description as a Reference for Conformity of Sequence Diagrams Using Text Mining (Case Study: SRS APTU)," 2022 IEEE World AI IoT Congress (AIoT), 2022.
- [11] S. Wu, "The Coexistence of Sequence Diagrams and Collaboration Diagrams in Unified Modeling Language," 2011 Fourth International Symposium on Knowledge Acquisition and Modeling, 2011, pp. 184-187, doi: 10.1109/KAM.2011.56.
- [12] P. K. Jayasekara and A. K.S., "Text Mining of Highly Cited Publications in Data Mining," 2018 5th International Symposium on Emerging Trends and Technologies in Libraries and Information Services (ETTLIS), 2018, pp. 128-130, doi: 10.1109/ETTLIS.2018.8485261.
- [13] G. Tereshchenko and I. Gruzdo, "Overview and Analysis of Existing Decisions of Determining the Meaning of Text Documents," 2018 International Scientific-Practical Conference Problems of Infocommunications. Science and Technology (PIC S&T), 2018, pp. 645-653, doi: 10.1109/INFOCOMMST.2018.8632014.
- [14] A. Azzam, Y. Priyadi and J. H. Husen, "Similarity Software Requirement Specification (SRS) Elicitation Based on the Requirement Statement Using Text Mining on the MNC Play Inventory Management Application," 2021 4th International Conference of Computer and Informatics Engineering (IC2IE), 2021, pp. 123-128, doi: 10.1109/IC2IE53219.2021.9649023.
- [15] R.P. Octavially, Y. Priyadi, and S. Widowati, "Extraction of Activity Diagrams Based on Steps Performed in Use Case Description Using Text Mining (Case Study: SRS Myoffice Application)," International Conference on Electrical and Electronic Inteligent System (ICE3IS), 2022.
- [16] S. A. Salloum, M. Al-Emran, A. A. Monem, and K. Shaalan, "Using text mining techniques for extracting information from research articles," 2018 In Studies in Computational Intelligence (Vol. 740, pp. 373–397), Springer Verlag, 2018, https://doi.org/10.1007/978-3-319-67056-0_18.
- [17] M. U. Salur and İ. Aydin, "The Impact of Preprocessing on Classification Performance in Convolutional Neural Networks for Turkish Text," 2018 International Conference on Artificial Intelligence and Data Processing (IDAP), 2018, pp. 1-4, doi: 10.1109/IDAP.2018.8620722.
- [18] Z. Yao and C. Ze-wen, "Research on the Construction and Filter Method of Stop-word List in Text Preprocessing," 2011 Fourth International Conference on Intelligent Computation Technology and Automation, 2011, pp. 217-221, doi: 10.1109/ICICTA.2011.64.
- [19] D. Soyusiawaty and Y. Zakaria, "Book Data Content Similarity Detector With Cosine Similarity (Case study on digilib.uad.ac.id)," 2018 12th International Conference on Telecommunication Systems Services and Applications (TSSA), 2018, doi:10.1109/tssa.2018.8708758.
- [20] J. A. Pamungkas, Y. Priyadi, and M. J. Alibasa, "Measurement of

- Similarity Between Requirement Elicitation and Requirement Specification Using Text Pre-Processing in the CINEMALOKA
- [21] T. A. Hutajulu, Y. Priyadi and A. Gandhi, "Text Data Processing in Requirement Specifications as a Reference for Similarities Between Use Case Diagrams and Use Case Descriptions for Smart Sleeping Lamp Application Documents," 2022 IEEE World AI IoT Congress (AIIoT), 2022.
 - [22] P. Kostakos, A. Pandya, O. Kyriakouli and M. Oussalah, "Inferring Demographic Data of Marginalized Users in Twitter with Computer Vision APIs," 2018 European Intelligence and Security Informatics Conference (EISIC), 2018, pp. 81-84, doi: 10.1109/EISIC.2018.00022.
 - [23] Z. Xie, C. Gadepalli and B. M. G. Cheetham, "Measurement of Rater Consistency by Chance-Corrected Agreement Coefficients," Application," 2022 IEEE World AI IoT Congress (AIIoT), 2022.
 - 2018 UKSim-AMSS 20th International Conference on Computer Modelling and Simulation (UKSim), 2018, pp. 14-20, doi: 10.1109/UKSim.2018.00015.
 - [24] Y. Priyadi, A. M. Putra and P. S. Lyanda, "The similarity of Elicitation Software Requirements Specification in Student Learning Applications of SMKN7 Baleendah Based on Use Case Diagrams Using Text Mining," 2021 IEEE 5th International Conference on Information Technology, Information Systems and Electrical Engineering (ICITISEE), 2021, pp. 115-120, doi: 10.1109/ICITISEE53823.2021.9655844.